KDD2026
August 9–13, 2026
Jeju, Korea

PYTHIA: Universal Question Answering over Knowledge Graphs

Sergios-Anestis Kefalidis[1], Konstantinos Plas[1,2] and Manolis Koubarakis[1,2]

[1]National and Kapodistrian University of Athens, Athens, Greece
[2]Athena Research Centre, Marousi, Greece

Paper

HELLENIC REPUBLIC
National and Kapodistrian University of Athens
EST. 1837

ATHENA
Research & Innovation Information Technologies

Code

Knowledge Graphs

Knowledge Graphs (KGs) provide rich semantic representations by modeling data as *nodes* and *relations* between them.

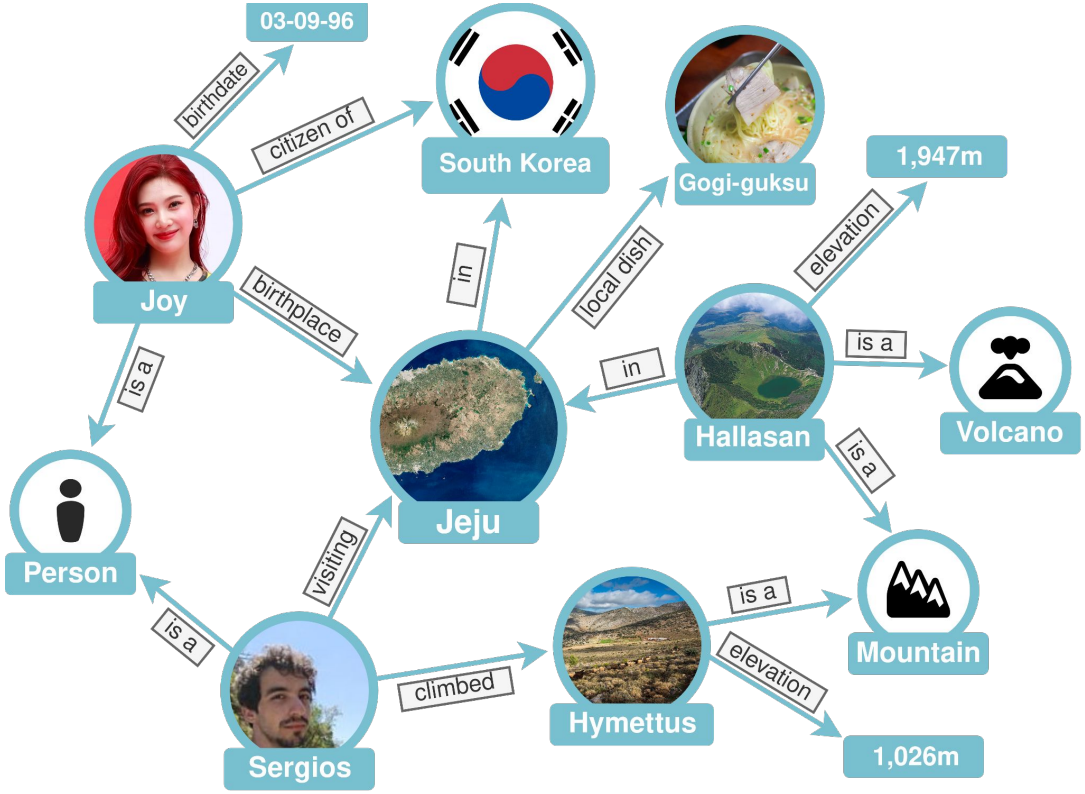
Note: In this work we are studying RDF graphs.

Useful to data engineers and end users

- Intuitive data model
- Easy integration of heterogeneous and incomplete data

Useful to LLMs

- Sources of factual information
- Graph model facilitates long-chain discovery and multi-hop reasoning



Knowledge Graphs

Knowledge Graphs (KGs) provide rich semantic representations by modeling data as *nodes* and *relations* between them.

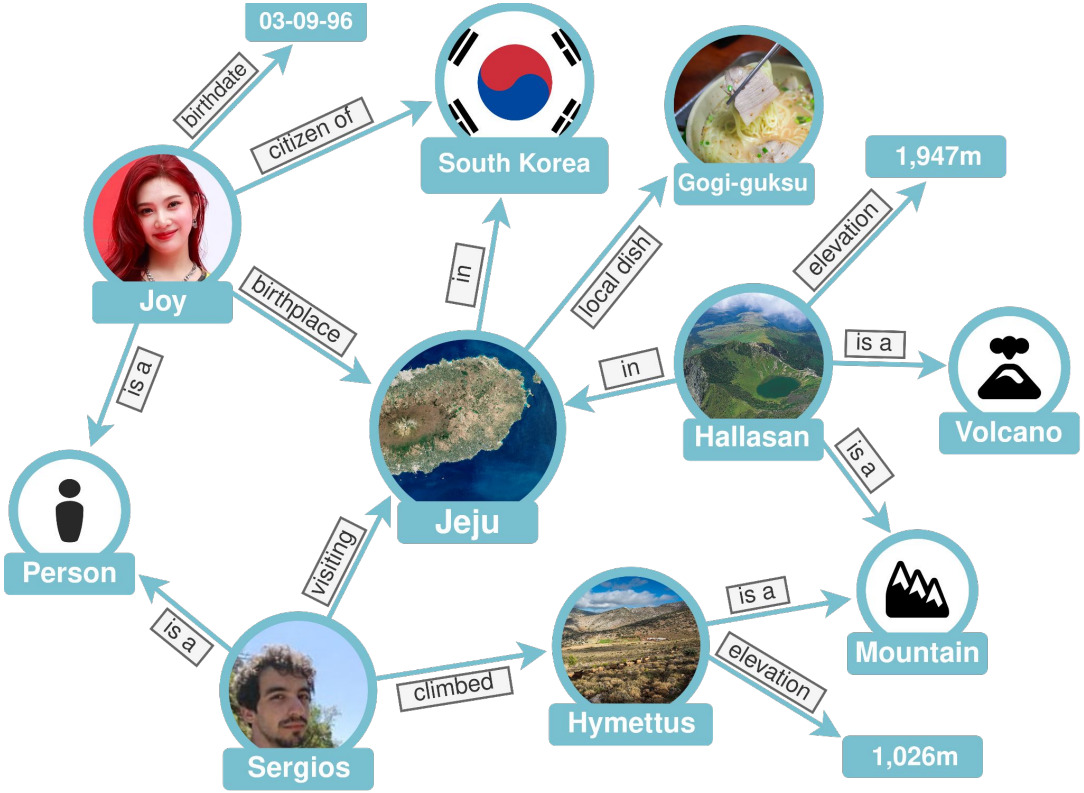
How do we access these information troves?

Users accessibility concerns

- **Know** SPARQL!
- **Know** the target KG's ontology and possible quirks!
- **Search** for topic entities!

LLM accessibility concerns

- Most KGs are too big to be given as context in their entirety!
- **Which information is relevant?**



Knowledge Graph Question Answering

Knowledge Graphs (KGs) provide rich semantic representations by modeling data as *nodes* and *relations* between them.

How do we access these information troves?

Users accessibility concerns



- **Know** SPARQL!
- **Know** the target KG's ontology and possible quirks!
- **Search** for topic entities!

LLM accessibility concerns



- Most KGs are too big to be given as context in their entirety!
- **Which information is relevant?**

KGQA — Addressing accessibility concerns



Knowledge Graph Question Answering (KGQA) answers natural language questions with information retrieved from a knowledge graph.

Design Decisions

01. Answer Generation Paradigm

How is the final answer generated?

02. Grounding Paradigm

How is the necessary information collected from the KG?

03. Control Flow Paradigm

What is the flow of information in the system?

Related Work: ChatKBQA

Answer Generation

Text-to-SPARQL

Translates natural language questions into structured SPARQL queries.

Grounding Paradigm

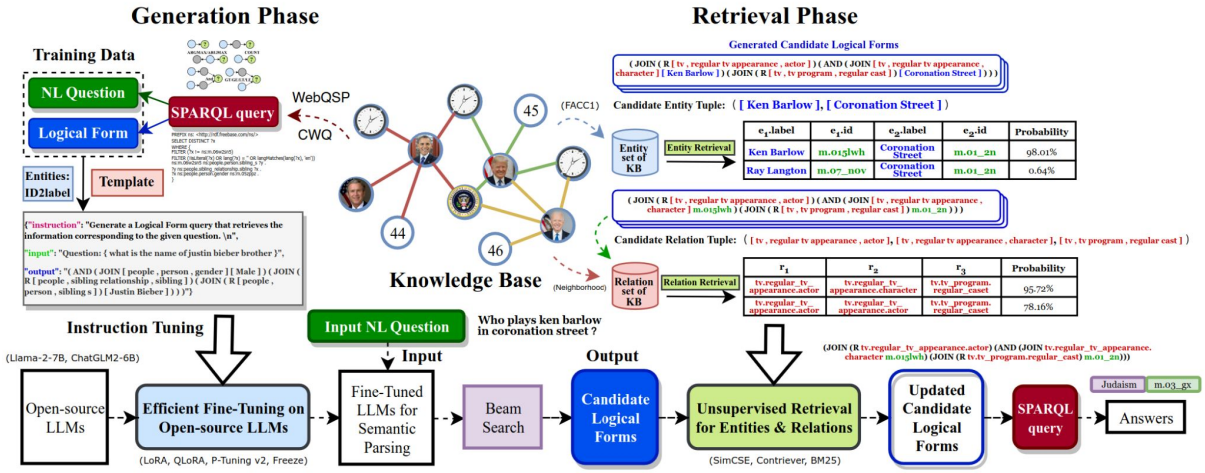
Direct Retrieval

Entity and relation retrieval are unsupervised, but aided by supervised fine-tuning of open-source LLMs through query sketches.

Control Flow

Pipeline Architecture

Executes structured operations sequentially in a distinct pipeline workflow.



ChatKBQA: A Generate-then-Retrieve Framework for Knowledge Base Question Answering with Fine-tuned Large Language Models. Luo et al. ACL Findings 2024

Related Work: Think-on-Graph

Answer Generation

KG-RAG

Information is utilized by an LLM which produces the final answer.

Grounding Paradigm

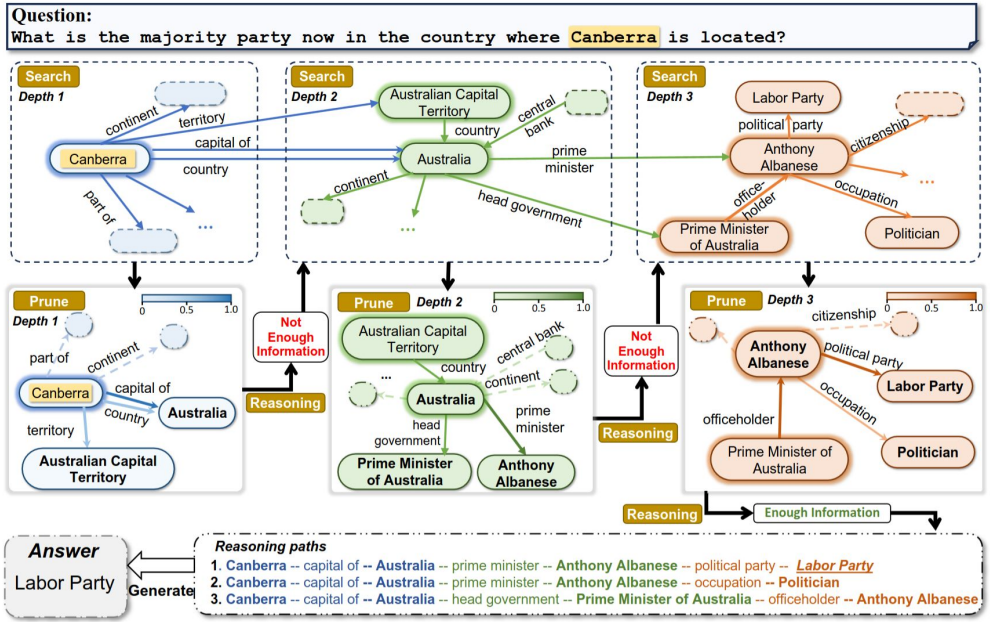
LLM-guided Exploration

Starting from specific nodes, the system iteratively explores the KG to retrieve useful information.

Control Flow

Pipeline Architecture

Executes structured operations sequentially in a distinct pipeline workflow.



Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. Sun et al. ICLR 2024.

Related Work: Agents for KGQA

Answer Generation

Various

Interactive-KBQA: Text-to-SPARQL
KG-Agent: Creates SPARQL-like “programs”

Grounding Paradigm

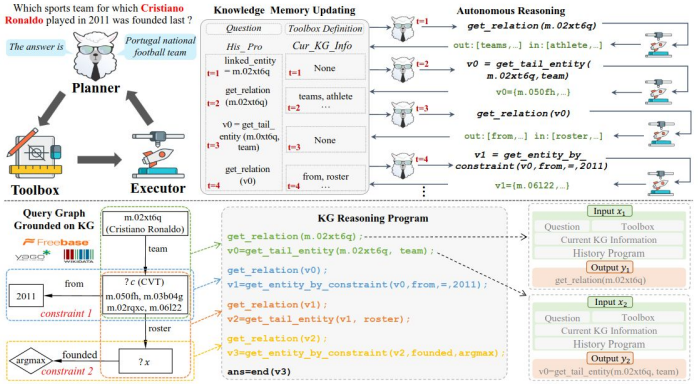
KG Exploration using Tools

Various tools provided enable interaction with the KG.

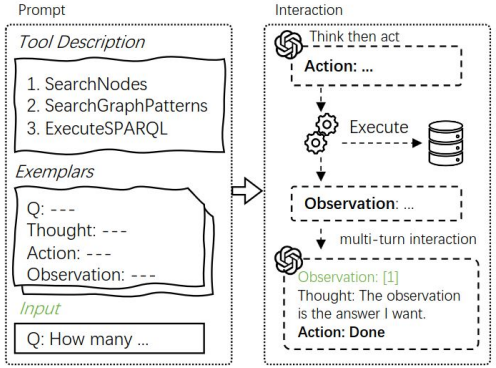
Control Flow

Agentic Architecture

An LLM equipped with tools attempts to iteratively provide a solution.



KG-Agent: An Efficient Autonomous Agent Framework for Complex Reasoning over Knowledge Graph. Jiang et al. ACL 2025.



Interactive-KBQA: Multi-Turn Interactions for Knowledge Base Question Answering with Large Language Models Reasoning over Knowledge Graph. Xiong et al. ACL 2024.

Knowledge Graph Question Answering: Challenges

Despite the great progress made by recent work, **4 core challenges** make the ‘holy grail’ of KGQA elusive:

01

Training-Free Execution

Requiring training data severely limits viability in real-world, purpose-built KGs.

02

Seamless Transferability

The ability to seamlessly deploy across different knowledge graphs without any modifications.

03

Generalizability

Robustly answering unseen questions outside the scope of training or development validation.

04

Accuracy vs Cost

Delivering highly precise, verified, and contextually grounded answers every time without making cost and response time explode.

PYTHIA

01 . Agentic Architecture

Iterative execution loop powered by LLMs equipped with tools to dynamically explore the knowledge graph.

02 . Text-to-SPARQL

Translates natural language questions directly into executable SPARQL queries to query databases.

03 . KG-Agnostic

Highly transferable design that deploys seamlessly across different knowledge graphs without modification.

04 . Training-Free

Operates fully in-context, avoiding the need for expensive training data or dedicated LLM fine-tuning.

PYTHIA: Exploration-Centric Framing

Any ***Agent*** consists of two parts: i) the LLM guided by a prompt and ii) the tools at its disposal. It follows that choosing an effective framing for the task is a crucial step in designing any agentic system.

However, prior work has generally overlooked this aspect, focusing instead on just tooling.

Limited Task Description

QueryAgent [1]

Provides generic task instructions with only examples as guidance.

Prompt:

```
You need to understand a question, and plan for the process of constructing a query to solve this question.

Examples follow...
```

Simple Iteration

SPINACH [2] & GRASP [3]

Describes iterative SPARQL construction by finding entities and properties.

Prompt (paraphrased):

```
You are a KGQA assistant. Your task is to understand the question, find entities and properties in the KG related to it and iteratively build up a SPARQL query.
```

Exploration-Centric Framing

PYTHIA

Framing KGQA explicitly as an exploration task across three stages:

i) Entity Linking
Identify named entities & link to KG nodes

ii) Property Retrieval
Retrieve properties of grounded nodes

iii) Semantic Connection Discovery
Explore relations between nodes:

- **Known-to-Unknown**
- **Known-to-Known**

Result A clear mental model for conducting entity linking and exploring the graph starting from linked nodes, without burdening it with the low-level execution details.

[1] *QueryAgent: A Reliable and Efficient Reasoning Framework with Environmental Feedback-based Self-Correction*. Huang et al. ACL 2024.
[2] *SPINACH: SPARQL-Based Information Navigation for Challenging Real-World Questions*. Liu et al. Find. of EMNLP 2024
[3] *GRASP: Generic Reasoning And SPARQL Generation across Knowledge Graphs*. Walter et al. ISWC 2025.

PYTHIA: Powerful Tooling with Simple Interfaces

Prior Art: Low-level chained steps, agent is responsible for every detail → **increase cognitive load on the Agent**.

PYTHIA: Powerful tools that complement the mental framing and **abstract the details of graph exploration**.

QueryAgent

get_relation

add_fact

add_max

add_min

add_count

add_filter

set_answer

execute

Interactive-KBQA

SearchNodes

SearchGraphPatterns

ExecuteSPARQL

GRASP

list

search_entity

search_property

search_property_of_entity

search_object_of_property

*search_autocomplete**

*search_constrained**

execute

PYTHIA

retrieve_entities_and_classes (i)

get_predicates_for_node (ii)

beam_search (iii)

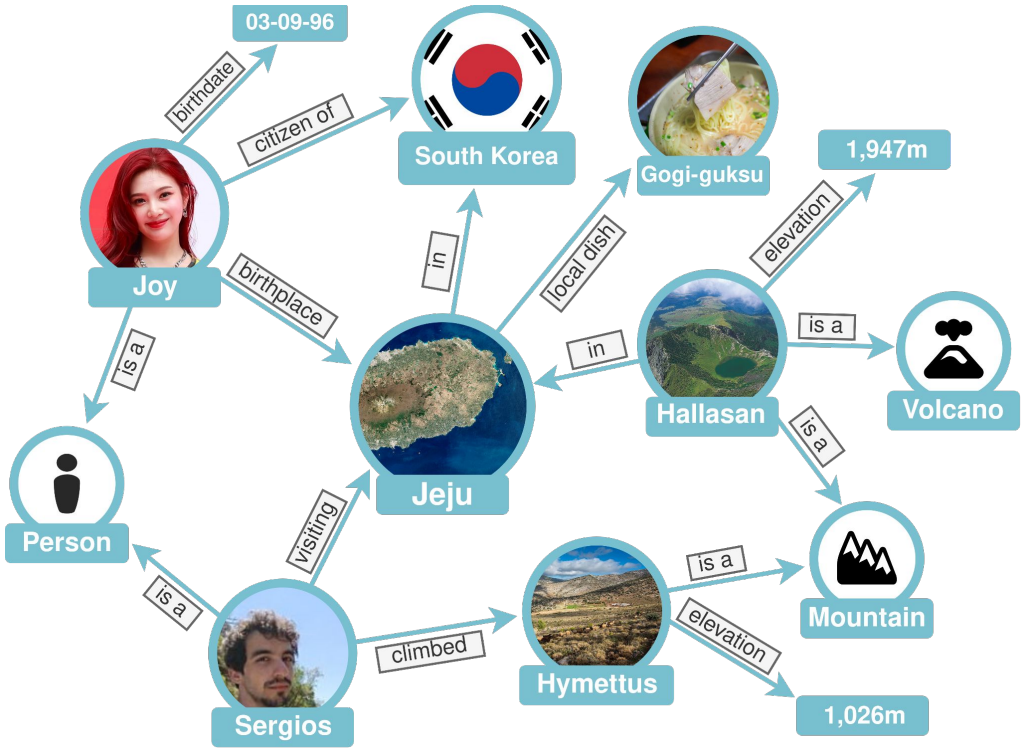
graph_search (iii)

execute_query

PYTHIA: Powerful Tooling with Simple Interfaces

Entity Linking (i) - retrieve_entities_and_classes

Question: **When was Joy born?**



PYTHIA: Powerful Tooling with Simple Interfaces

Entity Linking (i) - retrieve_entities_and_classes

Question: **When was Joy born?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Joy"*)



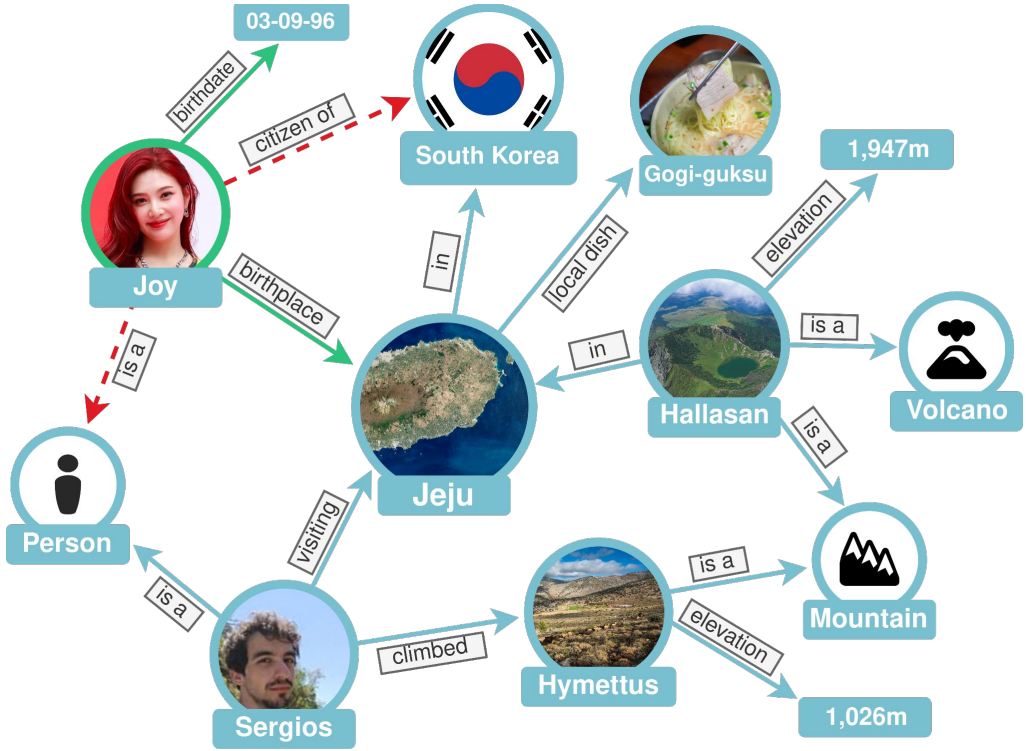
PYTHIA: Powerful Tooling with Simple Interfaces

Property Retrieval (ii) - get_predicates_for_node

Question: **When was Joy born?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Joy"*)

Step 2) Property retrieval for known entities
Tool call: ***get_predicates_for_node***(*kg:Joy, "date of birth"*)



PYTHIA: Powerful Tooling with Simple Interfaces

Query Execution and Response - execute_query

Question: **When was Joy born?**

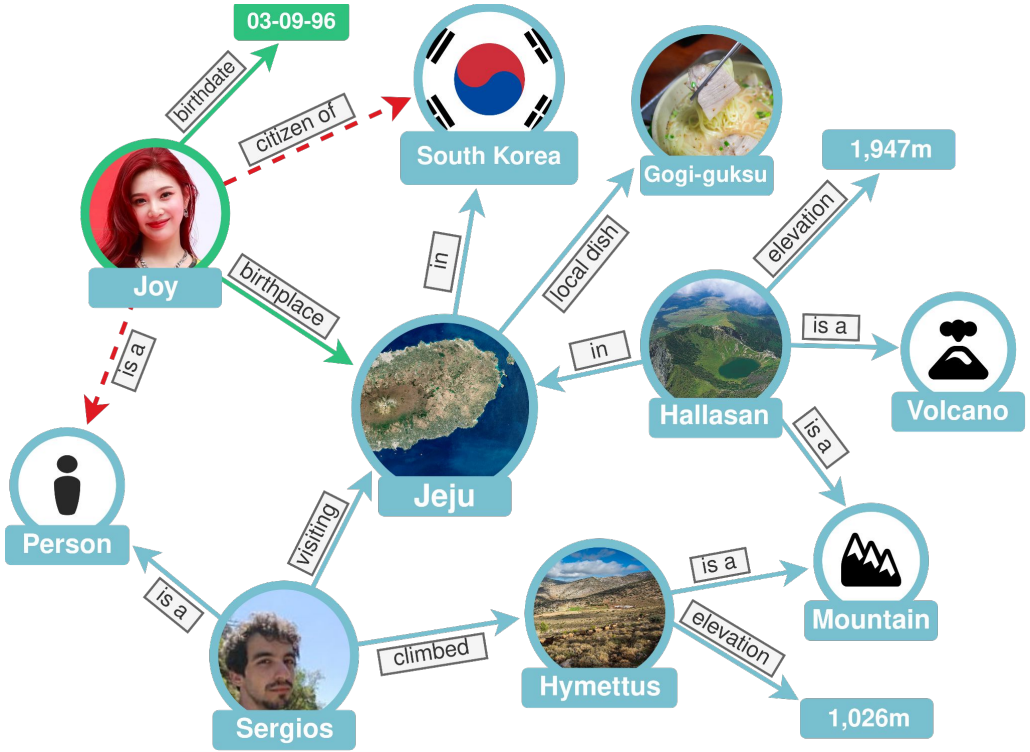
Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Joy"*)

Step 2) Property retrieval for known entities
Tool call: ***get_predicates_for_node***(*kg:Joy, "date of birth"*)

Step 3) Construct SPARQL query

```
SELECT ?date WHERE {
  kg:Joy kg:birthdate ?date
}
```

Tool call: ***execute_query***(*sparql_query*)



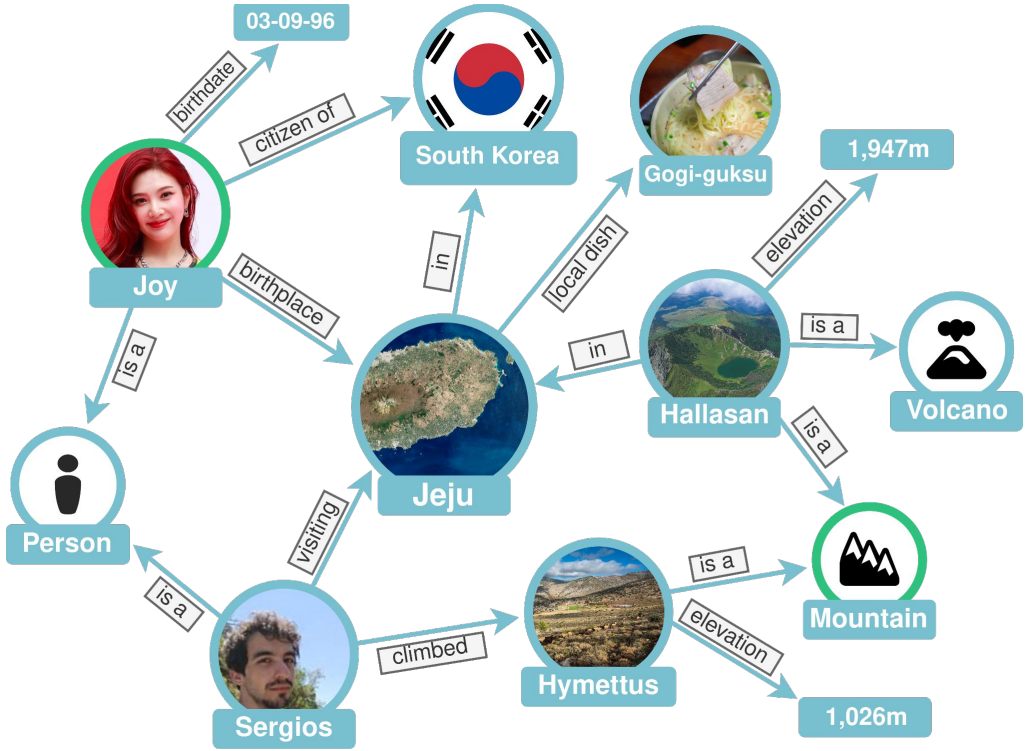
PYTHIA: Powerful Tooling with Simple Interfaces

Semantic Connection Discovery (iii) - graph_search

Question: **What mountains exist in Joy's birthplace?**

Step 1) Find topic entities (Entity Linking)

Tool call: ***retrieve_entities_and_classes***(*"Joy"*)
retrieve_entities_and_classes(*"Mountain"*)



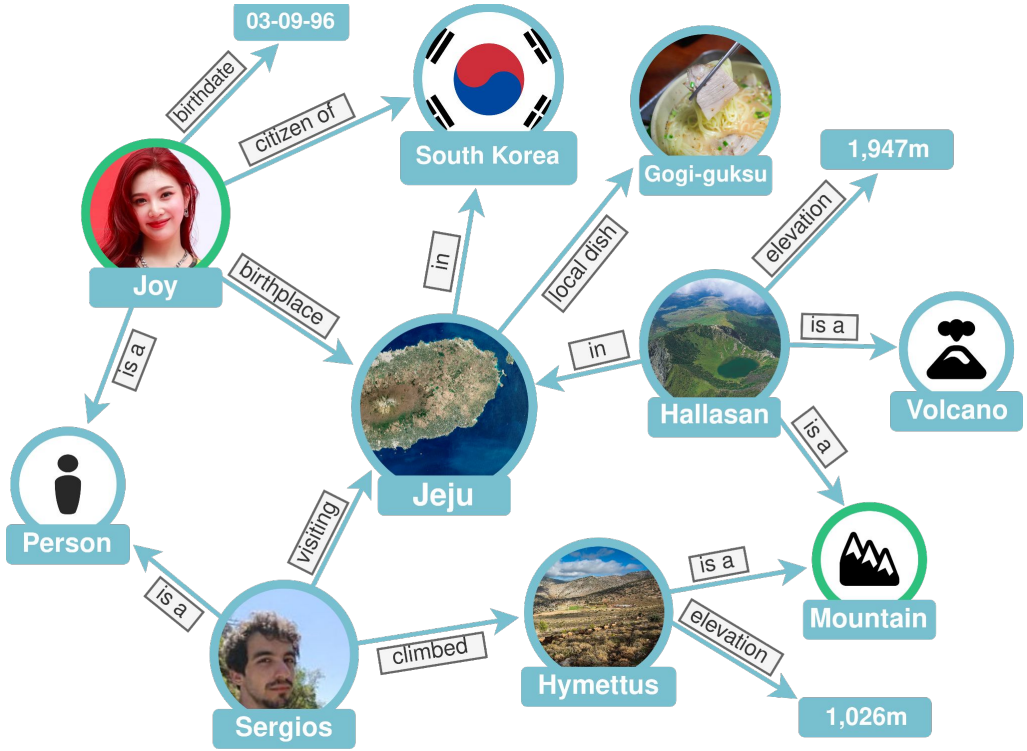
PYTHIA: Powerful Tooling with Simple Interfaces

Semantic Connection Discovery (iii) - graph_search

Question: **What mountains exist in Joy's birthplace?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Joy"*)
retrieve_entities_and_classes(*"Mountain"*)

Step 2) Iteratively search for paths between nodes?



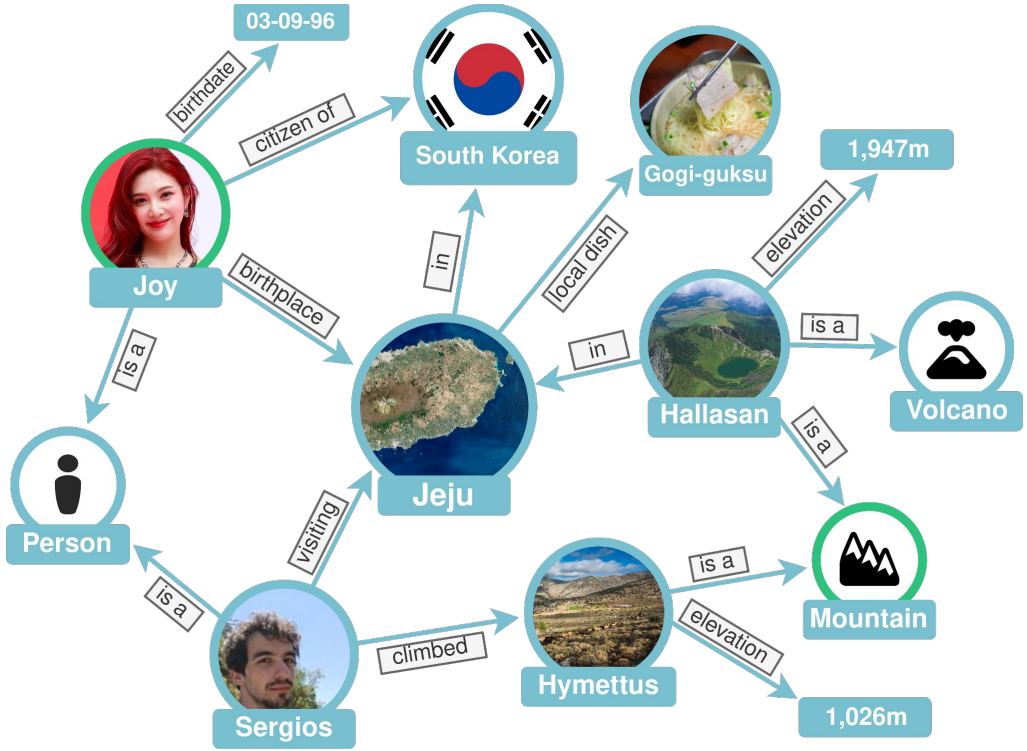
PYTHIA: Powerful Tooling with Simple Interfaces

Semantic Connection Discovery (iii) - graph_search

Question: **What mountains exist in Joy's birthplace?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Joy"*)
retrieve_entities_and_classes(*"Mountain"*)

Step 2) ~~Iteratively search for paths between nodes?~~



PYTHIA: Powerful Tooling with Simple Interfaces

Semantic Connection Discovery (iii) - graph_search

Question: **What mountains exist in Joy's birthplace?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Joy"*)
retrieve_entities_and_classes(*"Mountain"*)

Step 2) ~~**Iteratively search for paths between nodes?**~~

Step 2) Known-to-known connection discovery
Tool call: ***graph_search***(*kg:Joy, kg:Mountain, Question*)

How graph_search works internally:

i) Bidirectional Search: Find paths connecting the two target nodes in the KG.

ii) Cosine Filtering: Prune paths by calculating cosine similarity against the query string.

iii) LLM Selection: LLM evaluates which paths describe the query and those are selected for retrieval.

Optimizations: *Popularity & specificity help refine selection.*

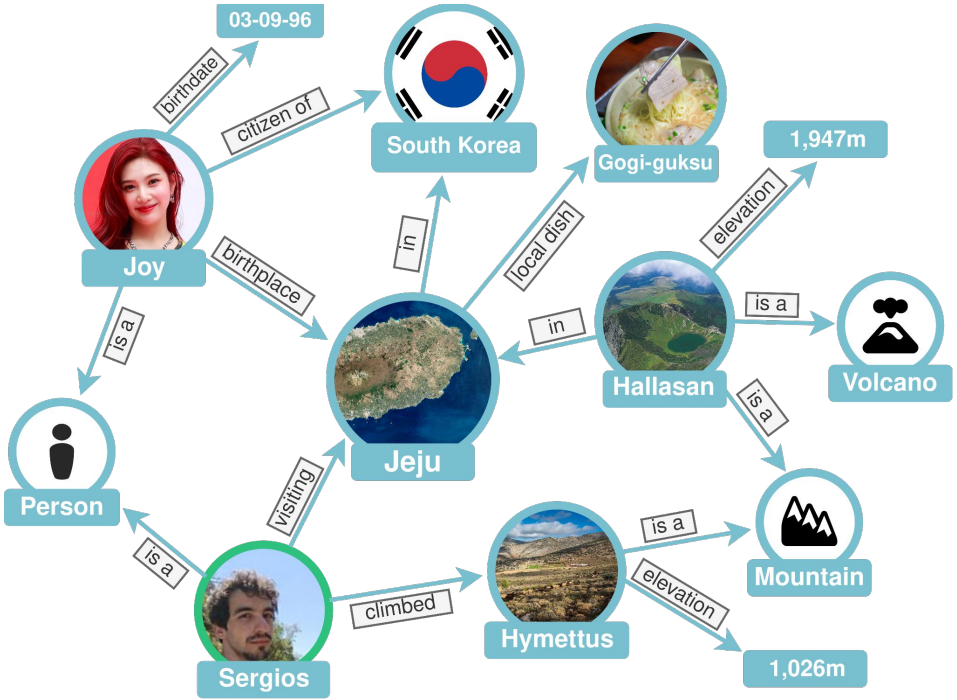


PYTHIA: Powerful Tooling with Simple Interfaces

Semantic Connection Discovery (iii) - beam_search

Question: **What local food can I (Sergios) try?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Sergios"*)



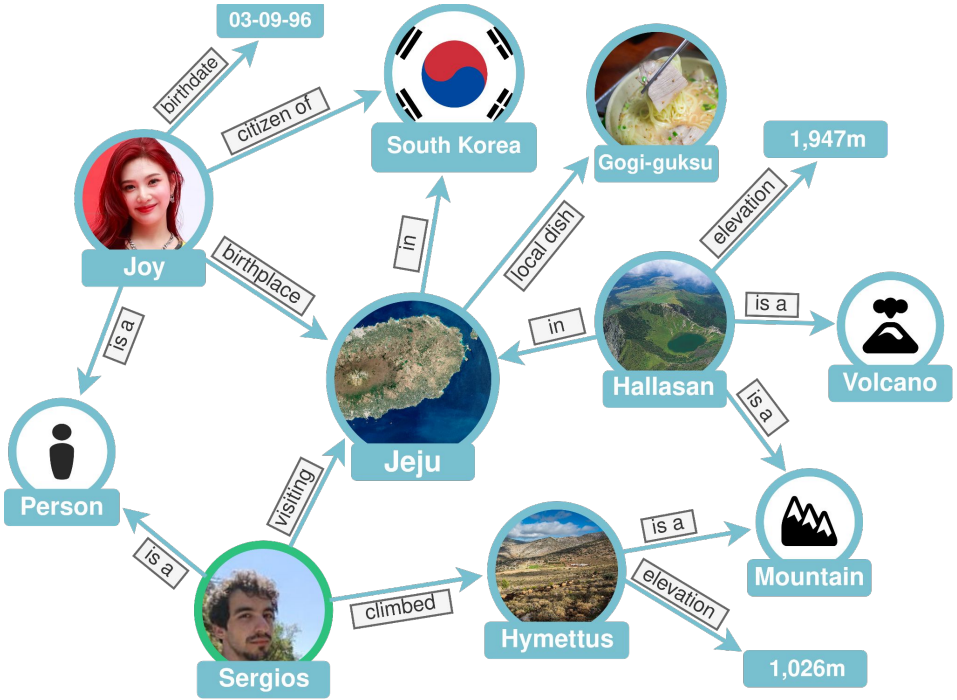
PYTHIA: Powerful Tooling with Simple Interfaces

Semantic Connection Discovery (iii) - beam_search

Question: **What local food can I (Sergios) try?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Sergios"*)

Target node is unknown and can't be linked.



PYTHIA: Powerful Tooling with Simple Interfaces

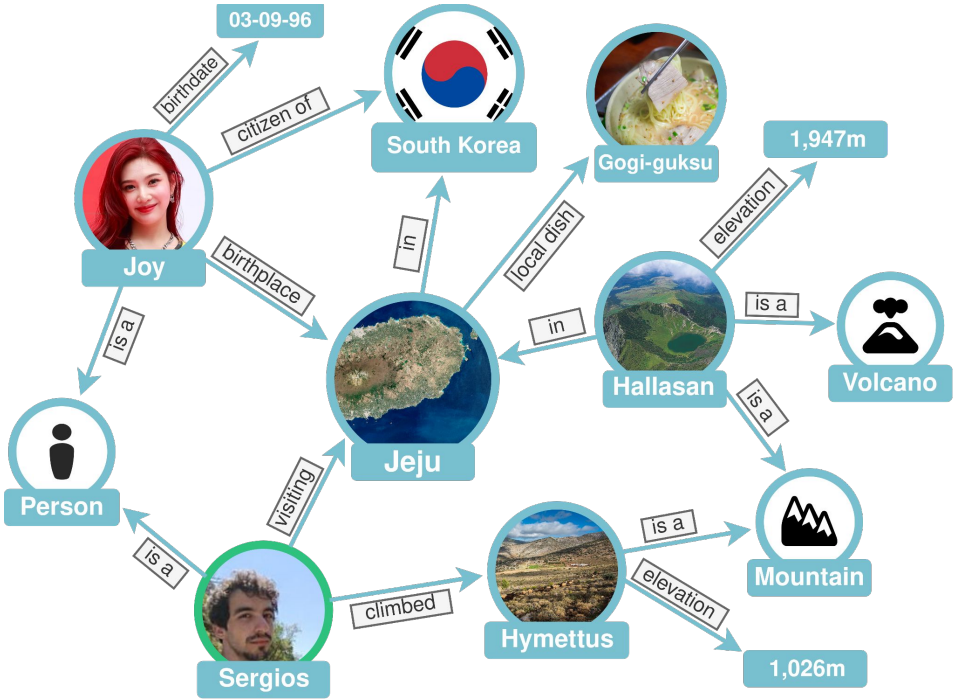
Semantic Connection Discovery (iii) - beam_search

Question: **What local food can I (Sergios) try?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Sergios"*)

Target node is unknown and can't be linked.

Iteratively search for target information.



PYTHIA: Powerful Tooling with Simple Interfaces

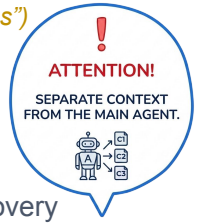
Semantic Connection Discovery (iii) - beam_search

Question: **What local food can I (Sergios) try?**

Step 1) Find topic entities (Entity Linking)
Tool call: ***retrieve_entities_and_classes***(*"Sergios"*)

Target node is unknown and can't be linked.

Iteratively search for target information.



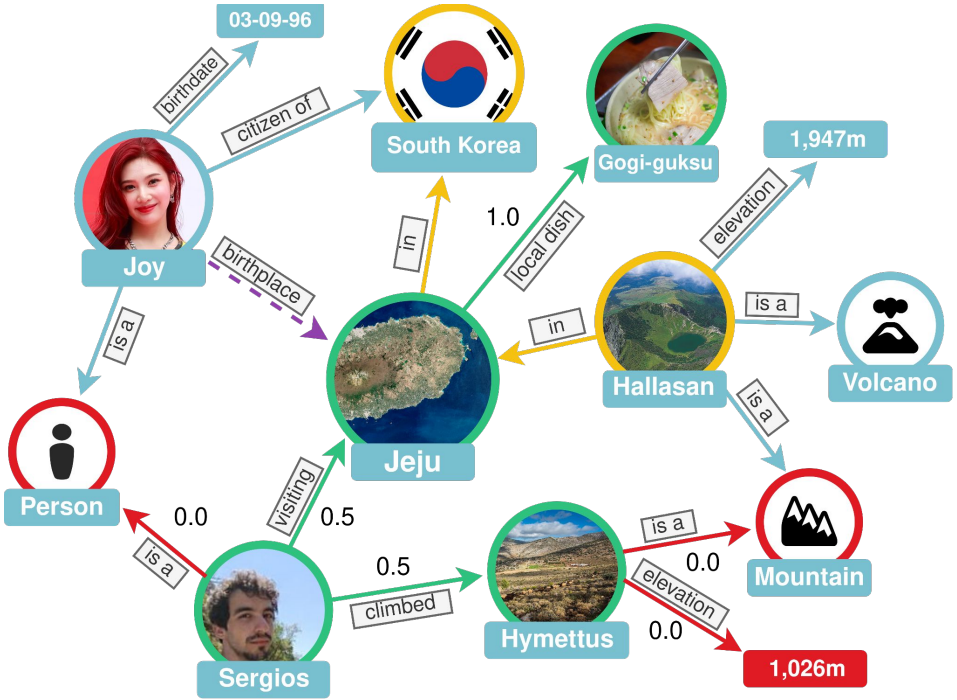
Step 2) Known-to-unknown connection discovery
Tool call: ***beam_search***(*kg:Sergios, "local dish for Sergios"*)

How beam_search works internally:

i) Expand Frontier: Expand predicates of frontier nodes, keeping only ***W*** (weighted pruning per node).

ii) LLM Decision: Remaining paths are sent to an LLM, which decides how to continue the search.

iii) Frontier Updated: The Frontier is updated, with the appropriate "super-nodes" being created.



Experiments

Datasets

We conduct our evaluation using **6 datasets** over **4 different KGs**. The selected datasets have varied characteristics to ensure appropriate coverage.

*A domain-specific dataset (**BESTIARY**) is used in addition to the usual encyclopedic datasets.*

Dataset	Knowledge Graph	Creation Method	Train/Test Overlap	SPARQL Complexity
WebQSP	Freebase	Semi-automatic	High	Low
CWQ	Freebase	Semi-automatic	High	Low
QALD-9	DBpedia	Manual	Low	Medium
LC-QuAD	DBpedia	Semi-automatic	Medium	Low
QALD-10	Wikidata	Manual	Low	Medium
BESTIARY	Custom KG	*Unknown*	**None**	**Higher**

Experiments

Systems

System	Algorithm	Implementation	Answer Method	Entity Linking	Architecture
ToG	**KG-agnostic**	Freebase & Wikidata	Direct	No	Pipeline
PoG	**KG-agnostic**	Freebase	Direct	No	Pipeline
GRASP	**KG-agnostic**	**KG-agnostic**	Semantic Parsing	**Yes**	Agentic
PYTHIA	**KG-agnostic**	**KG-agnostic**	Semantic Parsing	**Yes**	Agentic

System Selection

ToG [1] was selected as a highly influential, widely used baseline across the research area.

PoG [2] & GRASP [3] were chosen as they represent recent systems reporting state-of-the-art performance.

Implementation Upgrades

Because ToG and PoG do not support all KGs out of the box, we upgraded their implementations to be **KG-agnostic** and **triplestore-agnostic**.

These enhanced versions have been made fully available to the community.

Entity Linking

Since ToG and PoG lack native entity linking support, we integrated them with the **BELA [4]** entity linking system.

[1] *Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph*. Sun et al. ICLR 2024.
[2] *Paths-over-Graph: Knowledge Graph Empowered Large Language Model Reasoning*. Tan et al. WWW 2025.
[3] *GRASP: Generic Reasoning And SPARQL Generation across Knowledge Graphs*. Walter et al. ISWC 2025.
[4] *Multilingual End to End Entity Linking*. Plekhanov et al. Arxiv.

Experiments

Main Results - Zeroshot

System	WebQSP		CWQ		QALD-9		LC-QuAD		QALD-10		BESTIARY	
	F1	Hits@1	F1	Hits@1	F1	Hits@1	F1	Hits@1	F1	Hits@1	F1	Hits@1
LLM-Inherent	36.42	44.98	27.99	30.51	36.27	40.84	28.23	34.13	35.13	36.79	15.78	15.00
ToG	35.33	46.19	32.12	36.94	40.57	47.22	41.81	47.54	40.36	43.00	N/A	N/A
ToG*	43.76	56.63	36.22	39.87	42.40	50.00	46.55	53.65	41.87	44.52	19.93	21.00
PoG*	51.83	63.20	15.41	18.49	24.74	25.69	12.48	13.37	21.50	21.88	6.00	6.00
PoG* (Adjusted)	56.65	69.15	43.88	52.65	60.37	62.71	57.86	64.04	55.59	56.57	11.53	11.53
GRASP	65.79	68.56	44.10	45.57	60.88	64.58	44.39	46.64	64.98	64.12	42.66	43.00
GRASP*	64.36	68.85	44.40	45.98	64.39	66.66	48.65	50.45	66.38	65.90	41.38	41.00
PYTHIA	66.71	67.57	53.57	54.52	68.08	70.83	59.34	59.85	74.40	74.80	51.01	51.00
PYTHIA*	**73.99**	**75.30**	**61.62**	**62.96**	**73.08**	**74.30**	**67.10**	**67.36**	**76.11**	**76.59**	**51.99**	**52.00**

PYTHIA achieves **SotA** results in **zero-shot KGQA** across all 6 benchmarks (on Wikidata, DBpedia, Freebase and a D&D KG)

Experiments

Main Results - Dynamic Fewshot

System	WebQSP		CWQ		QALD-9		LC-QuAD		QALD-10		BESTIARY	
	F1	Hits@1	F1	Hits@1	F1	Hits@1	F1	Hits@1	F1	Hits@1	F1	Hits@1
LLM-SPARQL	28.19	26.98	27.69	27.29	7.11	6.25	31.07	31.33	2.14	1.52	N/A	N/A
LLM-SPARQL*	34.55	32.61	32.21	31.68	8.73	8.33	31.26	32.03	1.75	2.54	N/A	N/A
GRASP	71.54	73.20	59.20	60.21	55.81	57.63	55.37	56.15	60.62	59.79	N/A	N/A
GRASP*	76.44	77.72	65.60	66.54	66.27	71.52	64.92	66.66	70.57	70.22	N/A	N/A
PYTHIA	75.41	75.43	70.12	70.49	67.41	71.52	70.34	70.37	75.59	75.82	N/A	N/A
PYTHIA*	**80.69**	**80.63**	**75.14**	**75.42**	**76.82**	**79.16**	**76.94**	**76.77**	**77.63**	**78.37**	N/A	N/A

Dynamic Fewshot Setting

The **top-3 most similar** train-set question-query pairs are dynamically retrieved and provided as additional context to the model.

Baseline Comparison Note

ToG and **PoG** do not support dynamic few-shot prompting out of the box, making them unsuitable for direct comparison in this setting.

Experiments

Additional Results

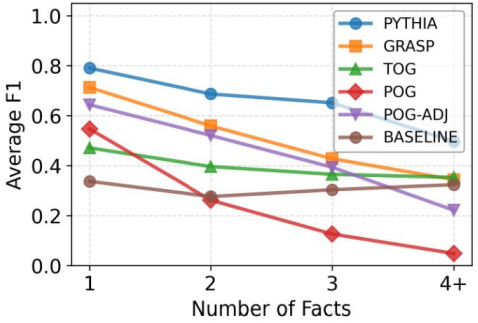
Table 3: PYTHIA compared to finetuned systems using F1.

	Model	WebQSP	CWQ
LLM-based (SFT)	Llama-2 7B* (ours)	80.29	80.60
	Pangu [9]	79.6	-
	RoG* [24]	70.8	56.2
	GCR [25]	74.1	61.7
	TFS-KBQA [44]	79.9	63.6
	SG-KBQA [8]	80.3	78.2
	ChatKBQA [23]	79.8	77.8
	ChatKBQA*	**83.5**	**81.3**
Ours (Few-shot)	**PYTHIA**	75.4	70.1
	PYTHIA*	80.7	75.1
	- Gap to SFT SotA	-3.3%	-7.6%

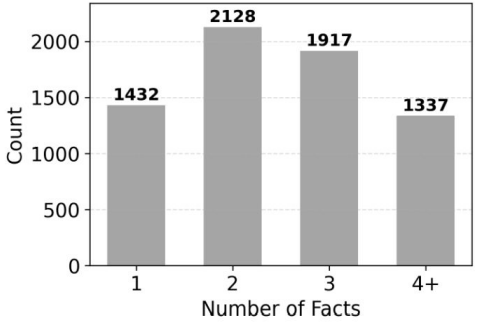
- PYTHIA with few-shot context **closes the gap to SFT**.
- These specific evaluation datasets are favourable to SFT approaches, as shown by the barebones **Llama-2 7B*** model which also manages SotA results.

Experiments

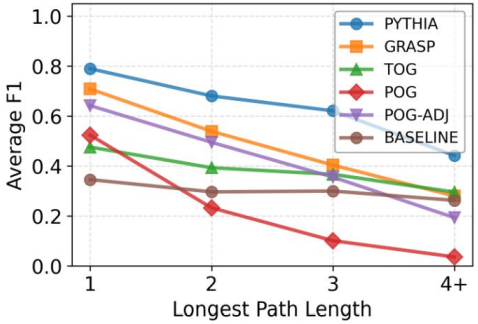
Analysis - Multi-hop Performance



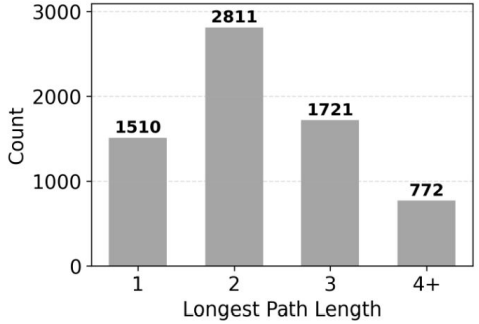
(a) Multi-Fact Performance



(b) Fact Count Distribution



(c) Path Length Performance



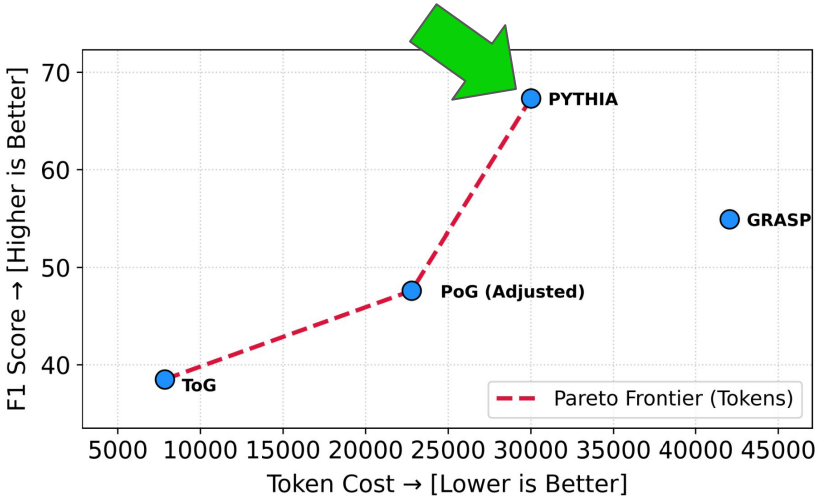
(d) Path Length Distribution

PYTHIA maintains long-chain performance better than competing systems.

Note: Baseline PoG collapse is attributed to query timeouts for multi-hop questions.

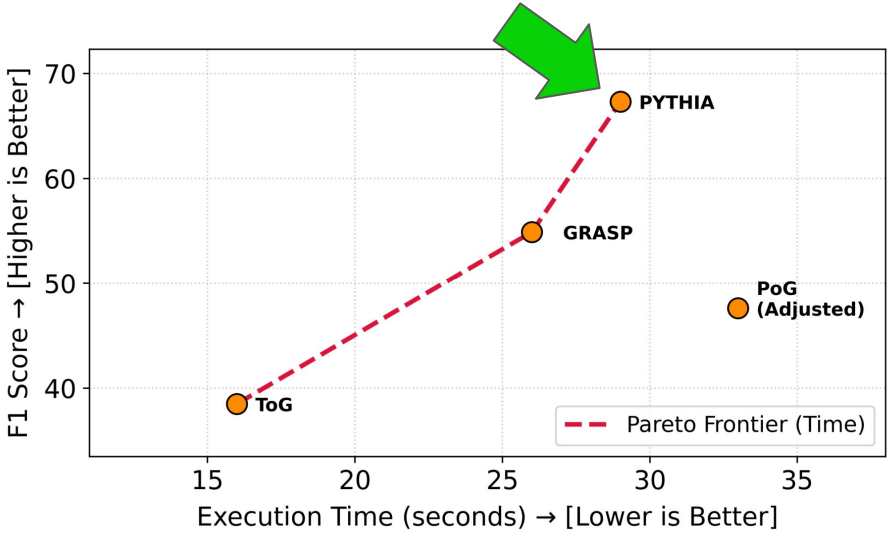
Experiments

Analysis - Efficiency



Leading token efficiency at the performance frontier!

*Token Cost = Input Tokens + 4 * Output Tokens*



Performance increase outweighs response time increase → Favourable trade-off

Thank you!

skefalidis@di.uoa.gr

Our Group

NLP for the Semantic Web, GeoAI, Entity Linking

My Links

KGQA, GraphRAG, GeoKGQA

Our group's LinkedIn

Our group's website

My Google Scholar

Aiteam

NATIONAL & KAPODISTRIAN UNIVERSITY OF ATHENS